



CONFIGURATION_GUIDE



HelixCode Configuration Guide

OVERVIEW

This comprehensive guide covers all aspects of HelixCode configuration through the `helix.json` file. Every service, component, and feature can be configured and managed through this central configuration system.

KEY FEATURES

- **Centralized Configuration:** All settings in `helix.json`
- **Dynamic Updates:** Configuration changes applied without restart
- **Validation:** Automatic validation of all configuration values
- **Hot Reload:** Real-time configuration updates
- **Environment Support:** Separate configs for different environments
- **Security:** Encrypted storage of sensitive data
- **Backup & Restore:** Automatic configuration backup
- **Version Control:** Track configuration changes over time

CONFIGURATION FILE STRUCTURE

Location

```
/helixcode/
├── helix.json           # Main configuration file
├── helix.backup.json    # Automatic backup
├── helix.template.json  # Template with defaults
├── configs/
│   ├── development.json # Development environment
│   ├── staging.json      # Staging environment
│   ├── production.json  # Production environment
│   └── local.json        # Local overrides
```

File Permissions

```
# Recommended permissions
chmod 600 helix.json           # Owner read/write only
```

```
chmod 644 helix.template.json      # Owner read/write, group/  
    others read  
chmod 700 configs/                 # Owner access only
```

COMPLETE CONFIGURATION STRUCTURE

```
{  
  "$schema": "./schemas/helix.schema.json",  
  "version": "1.0.0",  
  "environment": "production",  
  "debug": false,  
  "logging": {  
    "level": "info",  
    "format": "json",  
    "outputs": ["file", "console"],  
    "file": {  
      "path": "./logs/helix.log",  
      "max_size": "100MB",  
      "max_files": 10,  
      "rotate": true  
    }  
  },  
  "server": {  
    "mode": "production",  
    "host": "0.0.0.0",  
    "port": 8080,  
    "grpc_port": 9000,  
    "ssl": {  
      "enabled": true,  
      "cert_file": "./certs/server.crt",  
      "key_file": "./certs/server.key",  
      "auto_cert": true,  
      "ca_file": "./certs/ca.crt"  
    },  
    "cors": {  
      "allowed_origins": ["*"],  
      "allowed_methods": ["GET", "POST", "PUT", "DELETE"],  
      "allowed_headers": ["*"],  
      "max_age": 86400  
    },  
    "rate_limiting": {  
      "enabled": true,  
      "requests_per_minute": 1000,  
    }  
  }  
}
```

```
        "burst_size": 100,
        "whitelist": ["127.0.0.1"]
    },
    "database": {
        "primary": {
            "type": "postgresql",
            "host": "localhost",
            "port": 5432,
            "database": "helixcode",
            "username": "helixuser",
            "password": "${DB_PASSWORD}",
            "ssl_mode": "require",
            "max_connections": 100,
            "connection_timeout": "30s",
            "query_timeout": "60s",
            "pool_size": 20,
            "max_idle_time": "1h"
        },
        "cache": {
            "type": "redis",
            "host": "localhost",
            "port": 6379,
            "password": "${REDIS_PASSWORD}",
            "db": 0,
            "max_connections": 50,
            "default_ttl": "1h",
            "key_prefix": "helix:"
        }
    },
    "hardware": {
        "profiling": {
            "enabled": true,
            "auto_detection": true,
            "update_interval": "5m",
            "cache_duration": "1h"
        },
        "gpu": {
            "enabled": true,
            "memory_fraction": 0.8,
            "allow_growth": true,
            "multi_gpu": true,
            "visible_devices": "0,1"
        }
    },
```

```
"optimization": {
  "auto_tune": true,
  "performance_mode": "balanced",
  "cpu_affinity": true,
  "memory_management": "adaptive"
},
},
"cognee": {
  "enabled": true,
  "mode": "local",
  "host": "localhost",
  "port": 8000,
  "remote_api": {
    "service_endpoint": "https://api.cognee.ai",
    "api_key": "${COGNEE_API_KEY}",
    "timeout": "30s",
    "max_retries": 3,
    "retry_delay": "1s"
  },
  "local_config": {
    "data_path": "./data/cognee",
    "index_path": "./data/cognee/index",
    "max_size": 1000000,
    "max_memory": "2GB",
    "compression": true,
    "encryption": true
  },
  "performance": {
    "batch_size": 32,
    "max_concurrency": 10,
    "cache_size": 1000,
    "prefetch": true,
    "async_processing": true
  },
  "optimization": {
    "host_aware": true,
    "research_based": true,
    "auto_tune": true,
    "tune_interval": "1h"
  },
  "fallback": {
    "enabled": true,
    "strategy": "sequential",
    "timeout": "10s",
```

```
    "retry_count": 3,
    "providers": ["chromadb", "faiss", "redis"]
  },
  "security": {
    "encryption": true,
    "authentication": true,
    "authorization": true,
    "api_key_rotation": true,
    "rotation_interval": "24h"
  }
},
"memory": {
  "providers": {
    "chromadb": {
      "type": "chromadb",
      "enabled": true,
      "host": "localhost",
      "port": 8000,
      "path": "./data/chromadb",
      "api_key": "${CHROMADB_API_KEY}",
      "timeout": "30s",
      "max_retries": 3,
      "batch_size": 100,
      "compression": true,
      "metric": "cosine",
      "dimension": 1536
    },
    "pinecone": {
      "type": "pinecone",
      "enabled": false,
      "api_key": "${PINECONE_API_KEY}",
      "environment": "us-west1-gcp",
      "project_id": "my-project",
      "index_name": "helix-memory",
      "dimension": 1536,
      "metric": "cosine",
      "pod_type": "p1.x1",
      "pods": 1,
      "replicas": 1,
      "timeout": "30s",
      "max_retries": 3,
      "batch_size": 100,
      "namespace": "helix"
    }
  },
}
```

```
"faiss": {
  "type": "faiss",
  "enabled": true,
  "index_type": "ivf_flat",
  "dimension": 1536,
  "nlist": 100,
  "nprobe": 10,
  "metric": "cosine",
  "storage_path": "./data/faiss",
  "memory_index": true,
  "batch_size": 1000
},
"active_provider": "chromadb",
"health_monitoring": {
  "enabled": true,
  "interval": "30s",
  "timeout": "10s",
  "retry_count": 3,
  "alert_threshold": {
    "response_time": "500ms",
    "error_rate": 0.05,
    "success_rate": 0.95
  }
},
"fallback": {
  "enabled": true,
  "strategy": "sequential",
  "providers": ["chromadb", "pinecone", "faiss"],
  "timeout": "10s",
  "retry_count": 3
},
"load_balancing": {
  "strategy": "round_robin",
  "weights": {
    "chromadb": 1.0,
    "pinecone": 0.8,
    "faiss": 0.6
  }
},
"providers": {
  "openai": {
    "enabled": true,
```

```

"api_key": "${OPENAI_API_KEY}",
"organization": "${OPENAI_ORG_ID}",
"base_url": "https://api.openai.com/v1",
"timeout": "60s",
"max_retries": 3,
"retry_delay": "1s",
"models": {
  "default": "gpt-4",
  "chat": ["gpt-4", "gpt-4-turbo-preview", "gpt-3.5-turbo"],
  "completion": ["gpt-4", "gpt-3.5-turbo-instruct"],
  "embedding": ["text-embedding-3-large", "text-embedding-3-small"],
  "fine_tuned": ["gpt-4-0613", "gpt-3.5-turbo-0613"]
},
"rate_limits": {
  "requests_per_minute": 3500,
  "tokens_per_minute": 90000,
  "requests_per_day": 10000
},
"parameters": {
  "temperature": 0.7,
  "max_tokens": 4096,
  "top_p": 1.0,
  "frequency_penalty": 0.0,
  "presence_penalty": 0.0,
  "stream": false
}
},
"anthropic": {
  "enabled": true,
  "api_key": "${ANTHROPIC_API_KEY}",
  "base_url": "https://api.anthropic.com",
  "timeout": "60s",
  "max_retries": 3,
  "models": {
    "default": "claude-3-opus-20240229",
    "chat": ["claude-3-opus-20240229", "claude-3-sonnet-20240229", "claude-3-haiku-20240307"],
    "completion": ["claude-3-opus-20240229"]
  },
  "rate_limits": {
    "requests_per_minute": 1000,
    "tokens_per_minute": 50000
  }
}

```

```

    },
    "parameters": {
      "temperature": 0.7,
      "max_tokens": 4096,
      "top_p": 1.0,
      "top_k": 0,
      "stream": false
    }
  },
  "google": {
    "enabled": true,
    "api_key": "${GOOGLE_API_KEY}",
    "base_url": "https://generativelanguage.googleapis.com/v1beta",
    "timeout": "60s",
    "max_retries": 3,
    "models": {
      "default": "gemini-pro",
      "chat": ["gemini-pro", "gemini-pro-vision"],
      "embedding": ["embedding-001"]
    },
    "rate_limits": {
      "requests_per_minute": 60,
      "requests_per_day": 1500
    },
    "parameters": {
      "temperature": 0.7,
      "max_tokens": 4096,
      "top_p": 0.95,
      "top_k": 40,
      "stream": false
    }
  },
  "cohere": {
    "enabled": true,
    "api_key": "${COHERE_API_KEY}",
    "base_url": "https://api.cohere.ai/v1",
    "timeout": "60s",
    "max_retries": 3,
    "models": {
      "default": "command",
      "chat": ["command", "command-nightly", "command-light"],
      "embedding": ["embed-english-v3.0", "embed-multilingual-v3.0"]
    }
  }
}

```

```
    },
    "rate_limits": {
        "requests_per_minute": 1000,
        "requests_per_day": 10000
    },
    "parameters": {
        "temperature": 0.7,
        "max_tokens": 4096,
        "p": 0.75,
        "k": 0,
        "stream": false
    }
},
"replicate": {
    "enabled": true,
    "api_key": "${REPLICATE_API_TOKEN}",
    "base_url": "https://api.replicate.com/v1",
    "timeout": "60s",
    "max_retries": 3,
    "models": {
        "default": "meta/llama-2-70b-chat",
        "chat": ["meta/llama-2-70b-chat", "mistralai/
mixtral-8x7b-instruct"],
        "image": ["stability-ai/stable-diffusion", "lucataco/
realistic-vision"]
    },
    "parameters": {
        "temperature": 0.7,
        "max_tokens": 4096,
        "top_p": 0.9,
        "stream": false
    }
},
"huggingface": {
    "enabled": true,
    "api_key": "${HUGGINGFACE_API_KEY}",
    "base_url": "https://api-inference.huggingface.co",
    "timeout": "60s",
    "max_retries": 3,
    "models": {
        "default": "bigscience/bloom",
        "chat": ["bigscience/bloom", "microsoft/DialoGPT-
medium"],
        "embedding": ["sentence-transformers/all-MiniLM-L6-v2"]
    }
}
```

```

    },
    "parameters": {
        "temperature": 0.7,
        "max_tokens": 4096,
        "top_p": 0.9,
        "stream": false
    }
},
"vllm": {
    "enabled": true,
    "base_url": "http://localhost:8000/v1",
    "api_key": "${VLLM_API_KEY}",
    "timeout": "60s",
    "max_retries": 3,
    "models": {
        "default": "llama-2-7b-chat",
        "chat": ["llama-2-7b-chat", "mistral-7b-instruct"]
    },
    "parameters": {
        "temperature": 0.7,
        "max_tokens": 4096,
        "top_p": 0.9,
        "stream": false
    }
}
},
"api_keys": {
    "management": {
        "enabled": true,
        "encryption": true,
        "rotation_enabled": true,
        "rotation_interval": "24h",
        "backup_enabled": true,
        "storage": {
            "encrypted": true,
            "backup_path": "./secure/api_keys.backup"
        }
    }
},
"providers": {
    "openai": {
        "primary_keys": ["${OPENAI_API_KEY}"],
        "backup_keys": [],
        "auto_rotate": true,
        "rotation_interval": "7d",

```

```

    "rate_limit_tracking": true
  },
  "anthropic": {
    "primary_keys": ["${ANTHROPIC_API_KEY}"],
    "backup_keys": [],
    "auto_rotate": false,
    "rotation_interval": "30d"
  }
},
"security": {
  "authentication": {
    "enabled": true,
    "method": "jwt",
    "jwt": {
      "secret": "${JWT_SECRET}",
      "expiration": "24h",
      "refresh_expiration": "7d",
      "issuer": "helixcode",
      "algorithm": "HS256"
    },
  },
  "oauth": {
    "enabled": true,
    "providers": ["google", "github", "microsoft"],
    "google": {
      "client_id": "${GOOGLE_OAUTH_CLIENT_ID}",
      "client_secret": "${GOOGLE_OAUTH_CLIENT_SECRET}"
    },
    "github": {
      "client_id": "${GITHUB_OAUTH_CLIENT_ID}",
      "client_secret": "${GITHUB_OAUTH_CLIENT_SECRET}"
    }
  },
  "authorization": {
    "enabled": true,
    "rbac": {
      "enabled": true,
      "default_role": "user",
      "roles": {
        "admin": ["*"],
        "user": ["read", "write"],
        "guest": ["read"]
      }
    }
  }
}

```

```

    },
    "encryption": {
        "enabled": true,
        "algorithm": "AES-256-GCM",
        "key_derivation": "PBKDF2",
        "key_iterations": 100000,
        "data_at_rest": true,
        "data_in_transit": true
    },
    "rate_limiting": {
        "enabled": true,
        "global": {
            "requests_per_minute": 10000,
            "burst_size": 1000
        },
        "per_user": {
            "requests_per_minute": 1000,
            "burst_size": 100
        },
        "per_api_key": {
            "requests_per_minute": 5000,
            "burst_size": 500
        }
    },
    "audit_logging": {
        "enabled": true,
        "log_all_requests": true,
        "log_auth_events": true,
        "log_config_changes": true,
        "retention_days": 90
    }
},
"monitoring": {
    "enabled": true,
    "prometheus": {
        "enabled": true,
        "port": 9090,
        "path": "/metrics",
        "labels": {
            "service": "helixcode",
            "environment": "${ENVIRONMENT}"
        }
    }
},

```

```
"grafana": {
  "enabled": true,
  "url": "http://localhost:3000",
  "dashboards": [
    "./monitoring/grafana/dashboards/helixcode.json",
    "./monitoring/grafana/dashboards/memory.json"
  ]
},
"jaeger": {
  "enabled": true,
  "collector_url": "http://localhost:14268/api/traces",
  "service_name": "helixcode"
},
"health_checks": {
  "enabled": true,
  "interval": "30s",
  "endpoints": ["/health", "/ready", "/live"],
  "alert_threshold": {
    "response_time": "1s",
    "error_rate": 0.05
  }
}
},
"features": {
  "experimental": {
    "enabled": false,
    "features": ["quantum_search", "neural_interface"]
  },
  "beta": {
    "enabled": true,
    "features": ["voice_input", "image_generation",
      "code_completion"]
  },
  "stable": {
    "enabled": true,
    "features": ["text_completion", "memory_search",
      "conversation"]
  }
},
"plugins": {
  "enabled": true,
  "auto_load": true,
  "directories": ["/plugins", "/custom/plugins"],
  "configuration": {
```

```
    "update_check": true,
    "update_interval": "24h",
    "security_scan": true
  }
},
"development": {
  "debug_mode": false,
  "profiling": {
    "enabled": false,
    "port": 6060,
    "cpu_profiling": true,
    "memory_profiling": true,
    "block_profiling": true
  },
  "testing": {
    "mock_apis": false,
    "test_data_path": "./test/data",
    "parallel_tests": true
  },
  "hot_reload": {
    "enabled": false,
    "watch_paths": ["./internal", "./pkg"],
    "ignore_paths": ["./vendor", "./test"]
  }
},
"backup": {
  "enabled": true,
  "automatic": true,
  "schedule": "0 2 * * *",
  "retention": {
    "daily": 7,
    "weekly": 4,
    "monthly": 12
  },
  "storage": {
    "type": "local",
    "path": "./backups",
    "compression": true,
    "encryption": true,
    "remote": {
      "enabled": false,
      "type": "s3",
      "bucket": "helixcode-backups",
      "region": "us-west-2"
    }
  }
}
```

```
    }
  },
  "include": {
    "configuration": true,
    "database": true,
    "memory_data": true,
    "logs": false
  }
},
"notifications": {
  "enabled": true,
  "channels": {
    "email": {
      "enabled": true,
      "smtp_server": "smtp.gmail.com",
      "smtp_port": 587,
      "username": "${SMTP_USERNAME}",
      "password": "${SMTP_PASSWORD}",
      "from": "helixcode@example.com",
      "to": ["admin@example.com"]
    },
    "slack": {
      "enabled": true,
      "webhook_url": "${SLACK_WEBHOOK_URL}",
      "channel": "#helixcode",
      "username": "HelixCode Bot"
    },
    "discord": {
      "enabled": false,
      "webhook_url": "${DISCORD_WEBHOOK_URL}"
    }
  },
  "events": {
    "error": true,
    "warning": true,
    "info": false,
    "security": true,
    "maintenance": true
  }
}
}
```

CONFIGURATION SECTIONS DETAILED

1. CORE CONFIGURATION

Version & Environment

```
{  
  "version": "1.0.0",  
  "environment": "production",  
  "debug": false  
}
```

- **version:** Configuration format version
- **environment:** development, staging, production, test
- **debug:** Enable debug mode and verbose logging

Logging Configuration

```
{  
  "logging": {  
    "level": "info",  
    "format": "json",  
    "outputs": ["file", "console"],  
    "file": {  
      "path": "./logs/helix.log",  
      "max_size": "100MB",  
      "max_files": 10,  
      "rotate": true  
    }  
  }  
}
```

Options: - **level:** debug, info, warn, error, fatal - **format:** json, text, structured - **outputs:** file, console, syslog, elasticsearch

2. SERVER CONFIGURATION

Basic Server Settings

```
{  
  "server": {  
    "mode": "production",  
    "host": "0.0.0.0",
```

```
    "port": 8080,  
    "grpc_port": 9000  
  }  
}
```

SSL/TLS Configuration

```
{  
  "server": {  
    "ssl": {  
      "enabled": true,  
      "cert_file": "./certs/server.crt",  
      "key_file": "./certs/server.key",  
      "auto_cert": true,  
      "ca_file": "./certs/ca.crt"  
    }  
  }  
}
```

CORS Configuration

```
{  
  "server": {  
    "cors": {  
      "allowed_origins": ["*"],  
      "allowed_methods": ["GET", "POST", "PUT", "DELETE"],  
      "allowed_headers": ["*"],  
      "max_age": 86400  
    }  
  }  
}
```

3. DATABASE CONFIGURATION

Primary Database

```
{  
  "database": {  
    "primary": {  
      "type": "postgresql",  
      "host": "localhost",  
      "port": 5432,  
      "database": "helixcode",  
    }  
  }  
}
```

```

        "username": "helixuser",
        "password": "${DB_PASSWORD}",
        "ssl_mode": "require"
    }
}

```

Supported Types: - postgresql - mysql - sqlite - mongodb

Cache Configuration

```

{
  "database": {
    "cache": {
      "type": "redis",
      "host": "localhost",
      "port": 6379,
      "password": "${REDIS_PASSWORD}",
      "db": 0,
      "max_connections": 50
    }
  }
}

```

4. COGNEE CONFIGURATION

Basic Cognition Settings

```

{
  "cognition": {
    "enabled": true,
    "mode": "local",
    "host": "localhost",
    "port": 8000
  }
}

```

Modes: - local: Local deployment - hybrid: Local with cloud backup - cloud: Full cloud deployment

Remote API Configuration

```

{
  "cognition": {

```

```

    "remote_api": {
      "service_endpoint": "https://api.cognee.ai",
      "api_key": "${COGNEE_API_KEY}",
      "timeout": "30s",
      "max_retries": 3,
      "retry_delay": "1s"
    }
  }
}

```

Local Configuration

```

{
  "cognee": {
    "local_config": {
      "data_path": "./data/cognee",
      "index_path": "./data/cognee/index",
      "max_size": 1000000,
      "max_memory": "2GB",
      "compression": true,
      "encryption": true
    }
  }
}

```

5. MEMORY PROVIDER CONFIGURATION

ChromaDB Configuration

```

{
  "memory": {
    "providers": {
      "chromadb": {
        "type": "chromadb",
        "enabled": true,
        "host": "localhost",
        "port": 8000,
        "path": "./data/chromadb",
        "api_key": "${CHROMADB_API_KEY}",
        "timeout": "30s",
        "max_retries": 3,
        "batch_size": 100,
        "compression": true,

```

```

        "metric": "cosine",
        "dimension": 1536
    }
}
}
}

```

Pinecone Configuration

```

{
  "memory": {
    "providers": {
      "pinecone": {
        "type": "pinecone",
        "enabled": true,
        "api_key": "${PINECONE_API_KEY}",
        "environment": "us-west1-gcp",
        "project_id": "my-project",
        "index_name": "helix-memory",
        "dimension": 1536,
        "metric": "cosine",
        "pod_type": "p1.x1",
        "pods": 1,
        "replicas": 1,
        "namespace": "helix"
      }
    }
  }
}

```

6. PROVIDER CONFIGURATION

OpenAI Configuration

```

{
  "providers": {
    "openai": {
      "enabled": true,
      "api_key": "${OPENAI_API_KEY}",
      "organization": "${OPENAI_ORG_ID}",
      "base_url": "https://api.openai.com/v1",
      "timeout": "60s",
      "max_retries": 3,
    }
  }
}

```

```

"models": {
  "default": "gpt-4",
  "chat": ["gpt-4", "gpt-4-turbo-preview", "gpt-3.5-turbo"],
  "completion": ["gpt-4", "gpt-3.5-turbo-instruct"],
  "embedding": ["text-embedding-3-large", "text-embedding-3-small"]
},
"rate_limits": {
  "requests_per_minute": 3500,
  "tokens_per_minute": 90000,
  "requests_per_day": 10000
}
}
}
}

```

7. API KEY MANAGEMENT

API Key Configuration

```

{
  "api_keys": {
    "management": {
      "enabled": true,
      "encryption": true,
      "rotation_enabled": true,
      "rotation_interval": "24h",
      "backup_enabled": true
    },
    "providers": {
      "openai": {
        "primary_keys": ["${OPENAI_API_KEY}"],
        "backup_keys": [],
        "auto_rotate": true,
        "rotation_interval": "7d",
        "rate_limit_tracking": true
      }
    }
  }
}

```

8. SECURITY CONFIGURATION

Authentication

```
{
  "security": {
    "authentication": {
      "enabled": true,
      "method": "jwt",
      "jwt": {
        "secret": "${JWT_SECRET}",
        "expiration": "24h",
        "refresh_expiration": "7d",
        "issuer": "helixcode",
        "algorithm": "HS256"
      }
    }
  }
}
```

ENVIRONMENT VARIABLES

All sensitive data should be stored in environment variables:

```
# Database
export DB_PASSWORD="your_secure_password"
export REDIS_PASSWORD="your_redis_password"

# API Keys
export OPENAI_API_KEY="sk-your-openai-key"
export ANTHROPIC_API_KEY="sk-ant-your-key"
export GOOGLE_API_KEY="your-google-key"
export COHERE_API_KEY="your-cohere-key"
export REPLICATE_API_TOKEN="r8_your-token"
export HUGGINGFACE_API_KEY="hf_your-key"
export VLLM_API_KEY="your-vllm-key"

# Memory Providers
export PINECONE_API_KEY="your-pinecone-key"
export CHROMADB_API_KEY="your-chromadb-key"
export COGNEE_API_KEY="your-cognee-key"

# Security
```

```
export JWT_SECRET="your-jwt-secret"
export GOOGLE_OAUTH_CLIENT_ID="your-oauth-client-id"
export GOOGLE_OAUTH_CLIENT_SECRET="your-oauth-client-secret"
export GITHUB_OAUTH_CLIENT_ID="your-github-client-id"
export GITHUB_OAUTH_CLIENT_SECRET="your-github-client-secret"

# Monitoring
export SLACK_WEBHOOK_URL="https://hooks.slack.com/your-webhook"
export DISCORD_WEBHOOK_URL="https://discord.com/api/webhooks/
    your-webhook"

# SMTP
export SMTP_USERNAME="your-email@gmail.com"
export SMTP_PASSWORD="your-app-password"

# Development
export ENVIRONMENT="development"
```

CONFIGURATION MANAGEMENT

Loading Configuration

```
// Load configuration from helix.json
config, err := config.LoadFromFile("helix.json")
if err != nil {
    log.Fatal("Failed to load configuration:", err)
}

// Load configuration with environment overrides
config, err := config.LoadWithEnv("helix.json")
if err != nil {
    log.Fatal("Failed to load configuration:", err)
}

// Load configuration for specific environment
config, err := config.LoadForEnvironment("production")
if err != nil {
    log.Fatal("Failed to load production config:", err)
}
```

Updating Configuration

```
// Update configuration programmatically
err := config.UpdateKey("cognee.enabled", true)
if err != nil {
    log.Fatal("Failed to update configuration:", err)
}

// Update nested configuration
err := config.UpdateKey("providers.openai.models.default",
    "gpt-4-turbo-preview")
if err != nil {
    log.Fatal("Failed to update configuration:", err)
}

// Save configuration to file
err := config.SaveToFile("helix.json")
if err != nil {
    log.Fatal("Failed to save configuration:", err)
}
```

Validating Configuration

```
// Validate entire configuration
errors := config.ValidateAll()
if len(errors) > 0 {
    for _, err := range errors {
        log.Error("Configuration error:", err)
    }
    return fmt.Errorf("configuration validation failed")
}

// Validate specific section
errors := config.ValidateSection("cognee")
if len(errors) > 0 {
    return fmt.Errorf("cognee configuration is invalid")
}
```

CONFIGURATION SCHEMAS

JSON Schema

Create schemas/helix.schema.json for validation:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "HelixCode Configuration",
  "type": "object",
  "required": ["version", "server", "database", "cognee"],
  "properties": {
    "version": {
      "type": "string",
      "pattern": "^\\d+\\.\\d+\\.\\d+$"
    },
    "server": {
      "type": "object",
      "properties": {
        "host": {"type": "string"},
        "port": {"type": "integer", "minimum": 1, "maximum": 65535}
      },
      "required": ["host", "port"]
    },
    "cognee": {
      "type": "object",
      "properties": {
        "enabled": {"type": "boolean"},
        "mode": {"enum": ["local", "hybrid", "cloud"]}
      },
      "required": ["enabled", "mode"]
    }
  }
}
```

HOT RELOAD

Configuration changes are automatically detected and applied:

```
# Watch for configuration changes
```

```
./helixcode --config-watch
```

```
# Automatic reload interval (default: 5 seconds)
```

```
./helixcode --config-watch-interval 10s
```

```
# Configuration change events are logged
```

```
INFO Configuration file changed: ./helix.json
```

🔔 INFO Reloading configuration...
INFO Configuration reloaded successfully

SECURITY BEST PRACTICES

1. Environment Variables

- Never hardcode API keys or secrets
- Use strong, unique passwords
- Rotate keys regularly
- Use key management services in production

2. File Permissions

```
# Secure configuration file
chmod 600 helix.json
chown $USER:$USER helix.json
```

```
# Secure secrets directory
chmod 700 ./secrets
```

3. Backup and Version Control

```
# Add to .gitignore
echo "helix.json" >> .gitignore
echo "*.backup.json" >> .gitignore
echo "secrets/" >> .gitignore
```

```
📦 # Backup configuration
cp helix.json helix.backup.$(date +%Y%m%d_%H%M%S).json
```

CONFIGURATION CHECKLIST

Production Deployment Checklist

- ☐ Environment variables set for all secrets
- ☐ SSL certificates configured
- ☐ Database credentials secure
- ☐

- ☐ API key rotation enabled
- ☐ Rate limiting configured
- ☐ Monitoring enabled
- ☐ Backup schedule configured
- ☐ File permissions secure
- ☐ Configuration validated
- ☐ SSL/TLS enabled
- ☐ CORS properly configured
- ☐ Security headers set
- ☐ Audit logging enabled

Development Setup Checklist

- ☐
 - ☐ Debug mode enabled
 - ☐ Hot reload configured
 - ☐ Mock APIs set up
 - ☐ Test database configured
 - ☐ Profiling enabled
 - ☐ Development tools installed
 - ☐ IDE configuration applied
-

TROUBLESHOOTING

Common Issues

1. Configuration Not Loading

```
# Check file permissions
```

```
ls -la helix.json
```

```
# Validate JSON syntax
```

```
jq . helix.json
```

```
# Check schema validation
```

```
ajv validate -s schemas/helix.schema.json -d helix.json
```

2. Environment Variables Not Working

```
# Check if variables are set
```

```
env | grep OPENAI
```

```
# Source environment file
```

```
source .env
```

```
# Verify configuration loading
```

```
./helixcode --config-dry-run
```

3. Provider Connection Issues

```
# Test API key validity
```

```
curl -H "Authorization: Bearer $OPENAI_API_KEY" \
  https://api.openai.com/v1/models
```

```
# Check network connectivity
```

```
ping api.openai.com
```

```
# Verify DNS resolution
```

```
nslookup api.openai.com
```

ADDITIONAL RESOURCES

- [Configuration API Reference](#)
- [Environment Variables Guide](#)
- [Security Configuration](#)

- [Provider Configuration](#)
- [Migration Guide](#)

This guide is continuously updated. Check for the latest version at docs.helixcode.ai.